

Assembly

3CIIN

Maggio 2022

Fonti: Appunti Prof.Salvi,
Prof.ssa Secchi

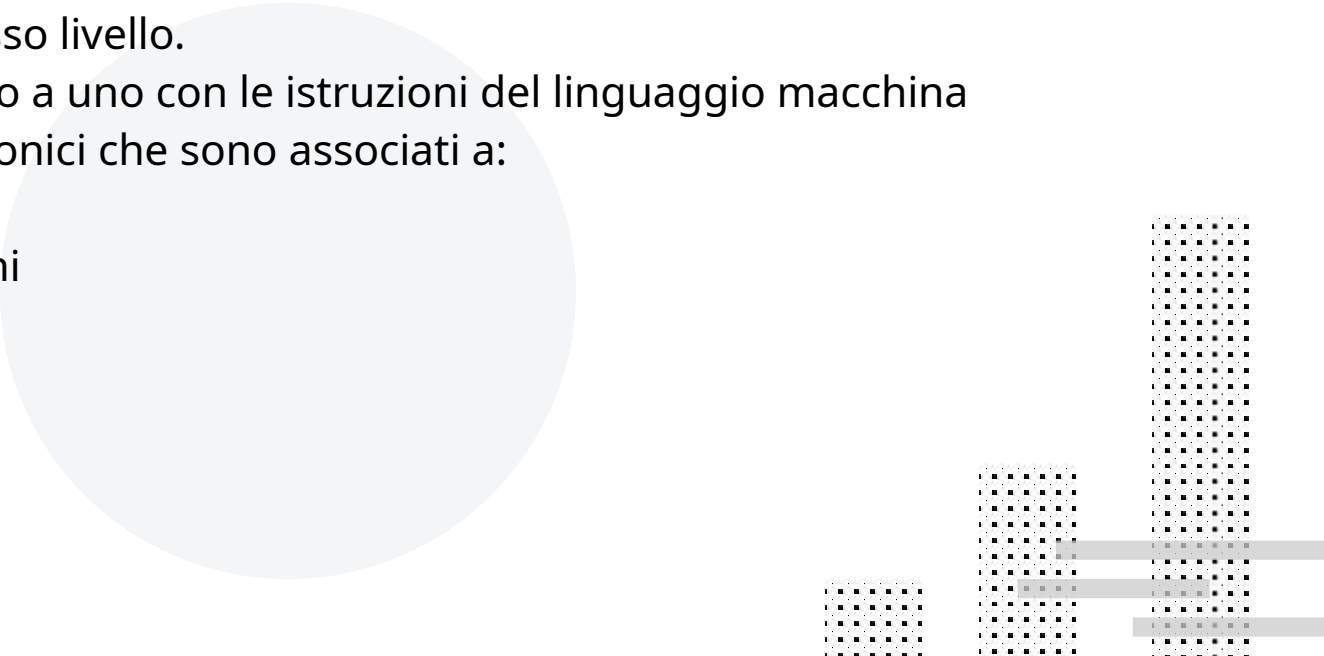


Caratteristiche del linguaggio Assembly

E' un linguaggio di basso livello.

C'è corrispondenza uno a uno con le istruzioni del linguaggio macchina

Utilizza simboli mnemonici che sono associati a:

- istruzioni
 - sequenze di istruzioni
 - indirizzi di memoria
 - aree di memoria
 - dispositivi di I/O
- 

Processore HC08

HC08 è un **Microcontrollore** (un sistema integrato) composto da:

- CPU a 8 bit
- ROM
- RAM 128 byte
- I/O
- Periferiche

Ha 64Kb di memoria disponibili (256 pagine di 256 celle)

I primi **64 bytes**, da 0x00 (0) a 0x3f (63) sono per **registri I/O**

128 bytes sono per la **RAM** tra la locazione 0x80 (128) e la 0xff (255)

4096 bytes per la **ROM programmabile** (Flash Eprom) da locazione 0xEE00 a 0xFDFF, 416 bytes per la ROM non riscrivibile tra le locazioni 0xFE10 e 0xFFAF,

I registri

Registro: è una variabile (può contenere un solo dato alla volta)

- Accumulator **A** memorizza valori e risultati di operazioni aritmetico-logiche
A → 8 bit
- Index **H:X** contiene indirizzi di memoria o indici di array
X → 8 bit Individua le prime 256 celle di memoria (0..255) e consente indirizzamento veloce (a pagina zero)
H:X → 16 bit Indirizza tutti i 64Kb di memoria
- Program Counter **PC** contiene indirizzo della prossima istruzione del programma
16 bit
- Stack Pointer **SP** contiene l'indirizzo della locazione di memoria occupata dal top dello stack
16 bit
- Conditioner Code **CCR** Contiene una serie di bit relativi al risultato dell'ultima operazione aritmetico/logica eseguita (come il riporto, la parità, il segno ecc.) o che influenzano il comportamento della CPU
8 bit

Istruzioni

Vengono tradotte dall'Assemblatore in istruzioni macchina. Ogni istruzione è composta in generale da:

- una Etichetta (Label)
- un Codice Operativo (Operation Code)
- uno o più operandi (Operands)

Esempio:

Label	Op.Code	Operands
START:	MOV	AX

Etichette (label)

Sono identificatori associati ad una istruzione; l'assemblatore le sostituisce con l'indirizzo dell'istruzione che rappresentano.

- permettono di trovare più facilmente un punto del programma
- facilitano la modifica del programma

Esempio:

Label
START:

Op.Code
MOV

Operands
#10, num1

Codici operativi (Op.code)

E' la stringa mnemonica dell'istruzione Assembly, specifica l'operazione che deve essere eseguita dalla CPU

Non può mai mancare in una riga di programma.

Esempio:

Label
START:

Op.Code
MOV

Operands
#10, num1

Operandi (Operands)

Possono non esserci.

Esempio:

Label
START:

Op.Code
MOV

Operands
#10, num1

Creazione di un programma Assembly

Il processo di creazione di un programma Assembly passa attraverso le seguenti fasi:

- scrittura del programma sorgente tramite un normale **EDITOR** in un file con estensione .asm
- assemblaggio, tramite un **ASSEMBLATORE** del file sorgente e generazione di un file oggetto con estensione .OBJ
- creazione del file eseguibile con estensione .EXE, tramite un **LINKER**

Struttura un programma Assembly

```
;ese.asm
; Descrizione del programma
; Costanti
STARTPGM    =    0xee00    ; L'inizio della ROM
STARTRAM    =    0x80     ; L'inizio della RAM
RESETVECT   =    0xffff   ; La posizione del vettore di reset
;L'area per le variabili
                .area    DATA (ABS)
                .org     STARTRAM
; Le definizioni delle variabili
; L'area del programma
                .area    PROGRAMMA (ABS)
                .org     STARTPGM
; Le istruzioni del programma
MAIN:        ; la "funzione" main
; il vettore di reset
                .area    RESET (ABS)
                .org     RESETVECT
                .word    MAIN    ; Puntatore a MAIN per il reset
```

Direttive
all'assemblatore

Programma assembly che esegue la somma CC di due valori AA e BB.

```
; i commenti sono preceduti da ";" non vengono considerati  
; ese.asm  
; somma AA + BB ed il risultato viene messo in CC
```

```
; COSTANTI che definiscono indirizzi di memoria da cui partono:
```

```
STARTPGM    = 0xee00          ; ipotetica ROM (flash)  
RESETVECT   = 0xffff         ; vettore di reset (punt a prima riga da eseguire)  
STARTDATI   = 0xb1000000      ; Area dati (primo indirizzo di RAM  
                               ; disponibile) in esad = 80 in dec = 128
```

```
; definisco area che chiamo DATI  
; per raggruppare i dati (VARIABILI e Procedure)
```

```
    .area DATI (ABS)  
    .org STARTDATI          ; indirizzo da cui parte area di definizione DATI  
  
AA:      .blkb      1      ; definisco tre variabili AA BB CC (alloco spazio)  
BB:      .blkb      1      ; blkb definisce blocco di n bytes non iniz.  
CC:      .blkb      1      ; AA BB CC occupano ciascuno 1 bytes
```

; definisco area che chiamo PROGRAMMA per raggruppare le **ISTRUZIONI** operative

```
.area    PROGRAMMA (ABS)
.org     STARTPGM          ; inizio area PROGRAMMA

MAIN: LDA #0b10000001      ;LoaDA mette in A (registro accumulatore) il valore 129
    STA AA                 ;SToreA mette in AA il valore che trova
                           ;nell'accumulatore A --> AA=129

    LDA #1                 ;LoaDA mette in A (registro accumulatore) il valore 1
    STA BB                 ;SToreA mette in BB il valore che trova
                           ;nell'accumulatore A --> BB=1

    LDA AA                 ;LoaDA mette in A (registro accum.) il valore AA (129)
    ADD BB                 ;aggiunge il valore BB (1) in A (registro accumulatore)
                           ;che avrà quindi valore 130
    STA CC                 ;SToreA mette in CC il valore che trova
                           ;nell'accumulatore A --> CC=130 (CC=AA+BB)

FINE: BRA FINE             ;Loop infinito per fermare il programma
```

; definisco area che chiamo RESET per raggruppare le istruzioni di reset

```
.area    RESET (ABS)
.org     RESETVECT
.word    MAIN              ; alloca 2 byte con valore MAIN (locazione iniz. del prg)
```

Definizione costanti*

NELEM = 10 costante decimale

MASK = 0b01010101 costante binaria

STARTDATI = 0x0080 costante esadecimale

*Inserire il codice prima di tutte le aree definite.

Definizione variabili e procedure* (allocazione celle della RAM)

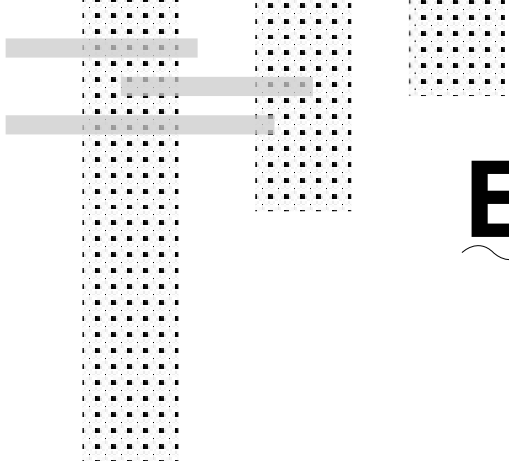
MAX: .blkb 1

variabile con un blocco di 1 byte (non inizializzata)

VETT: .blkb 8

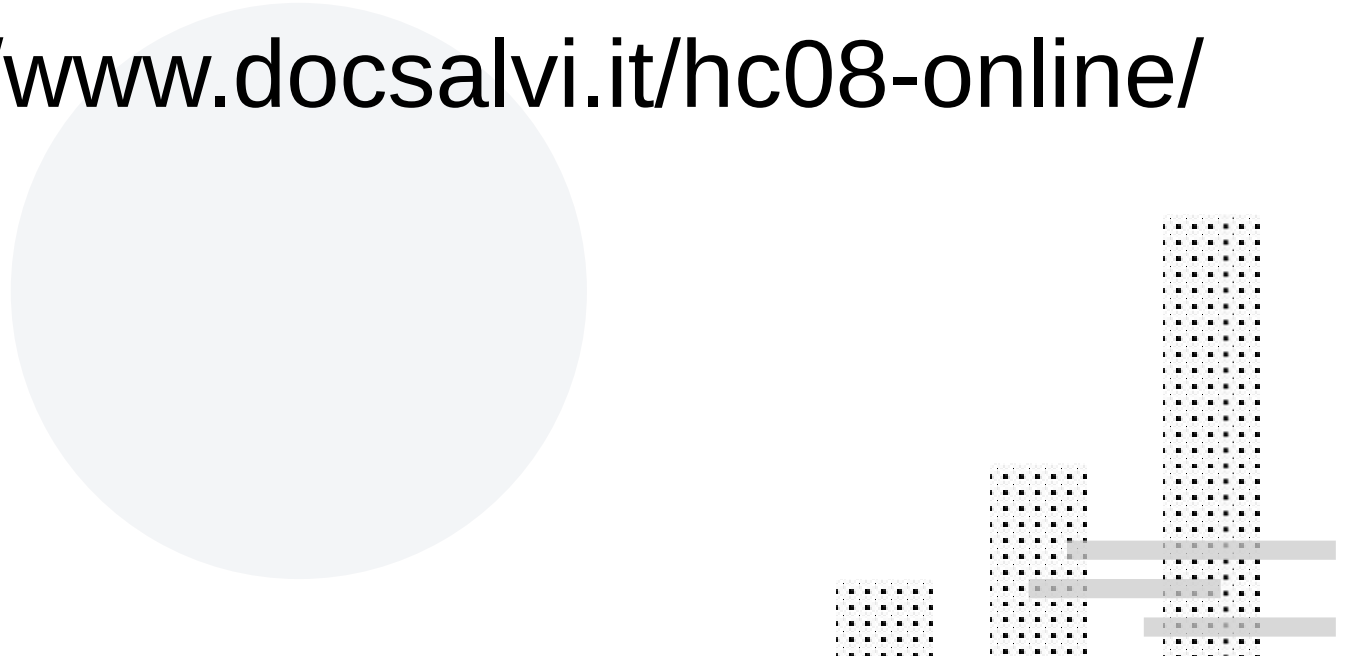
un vettore con 8 blocchi (celle) da 1 byte ciascuno

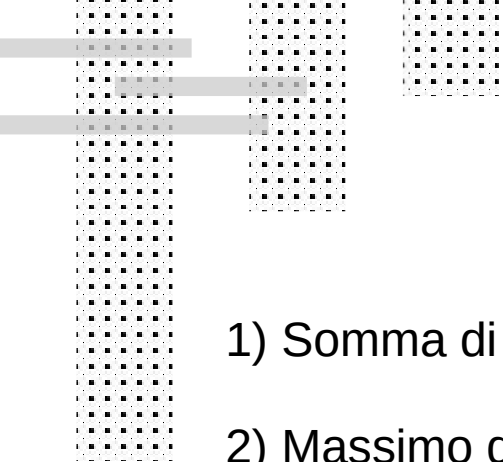
*Inserire il codice nell'area dati (RAM) riservata



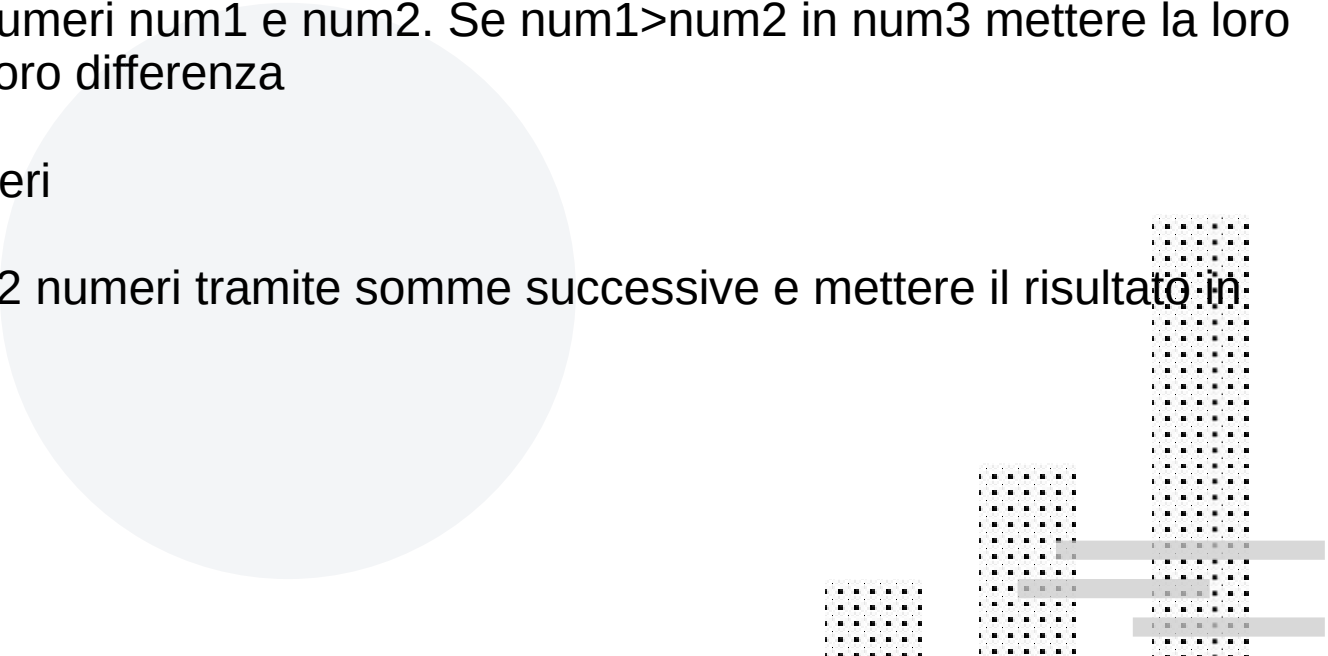
Emulatore HC08

<https://www.docsalvi.it/hc08-online/>





Esercizi

- 1) Somma di 3 numeri
 - 2) Massimo di 2 numeri
 - 3) Confrontare due numeri num1 e num2 . Se $\text{num1} > \text{num2}$ in num3 mettere la loro somma altrimenti la loro differenza
 - 4) Massimo di 3 numeri
 - 5) Fare il prodotto di 2 numeri tramite somme successive e mettere il risultato in num3
- 

IF

<pre><u>If</u> (A<M){ a } b c</pre>	<pre>SE: <u>CMP</u> M <u>BGE</u> MORE a MORE: b c</pre>	Compara A con M (A-M) e se $A \geq M$, salta a label MORE ed esegue b e c, altrimenti esegue istruzioni a e poi b c La condizione va <u>invertita</u> rispetto al C
<pre><u>If</u> (A>M){ a }else{ b } c</pre>	<pre>SE: <u>CMP</u> M <u>BLE</u> ELSE a BRA <u>END_IF</u> ELSE: b <u>END_IF</u>: c</pre>	Compara A con M (A-M) e se $A \leq M$, salta a label ELSE ed esegue b e c, altrimenti esegue istruzioni a e c La condizione va <u>invertita</u> rispetto al C

WHILE

```
While (A==M) {
```

```
    istruz
```

```
}
```

```
LOOP: CMP M
```

```
    BNE(**) FINE
```

```
    istruz
```

```
    BRA LOOP
```

```
FINE:
```

```
** BEQ
```

```
** BNE
```

```
** BLE
```

```
** BLT
```

```
** BGE
```

```
** BGT
```

Compara (CoMPare) valore del registro A con valore di M

se A!=M Branch (salta) a FINE

altrimenti (A==M) ed esegue istruz

torna a LOOP

A=M Equal

A!=M NotEqual

A<=M LessOrEqual

A<M LessThan

A>=M GreaterOrEqual

A>M GreaterThan

DO-WHILE

```
Do{  
    istruz  
} while (A==M);
```

```
LOOP:  
    istruzione  
  
    CMP M  
    BEQ LOOP
```

Esegue istruzioni,

compara (CoMPare) A con M e
se A==M torna su a label LOOP
altrimenti prosegue

FOR

```
for (I=1; I<N; I++){  
    istruz  
}
```

```
MOV #1, I  
LDA I  
FOR:  
  
CMP #N, FINEFOR  
  
BGE FINEFOR  
    istruz  
    INC I  
    LDA I  
    BRA FOR  
FINEFOR:
```

Memorizza il valore 1 nella variabile I e carica in A il valore di I; I=1

Compara A con il valore cost N (confronta I con N)
Se I>=N esce dal ciclo (va a FINEFOR - il contrario rispetto al C), altrimenti se I<N esegue istruz, incrementa I e ritorna al controllo (label FOR)